Utilisation des tests sur les composants React

1. Pourquoi tester une application React ?

Quand on développe une application React, elle peut fonctionner aujourd'hui… mais casser demain après une petite modification.
Les tests permettent donc :

- ✓ Vérifier que les composants affichent les bonnes informations
- ✓ Vérifier que les composants réagissent correctement aux clics / formulaires
- ✓ Éviter que les futures modifications cassent l'application (régression)

Les tests sont une sécurité pour le développeur.

2. Que tester exactement dans React ?

Dans le Front-end, on ne teste pas le design, ni les couleurs, ni le CSS.
On teste les choses utiles, par exemple :

✓ Ce qui s'affiche

- Le texte
- Les boutons

- Les éléments dynamiques

✓ **Les interactions**

- Cliquer sur un bouton
- Remplir un champ
- Vérifier que l'état (state) change

✓ **Le comportement du composant**

- Afficher un message selon une condition
- Exécuter une fonction passée en props

3. L'environnement de test (Jest)

React utilise Jest, un outil de test fourni automatiquement avec Create React App.

Un test ressemble à ceci :

```
describe("nom du test", () => {
  it("ce que le test doit vérifier", () => {
    // test ici
  })
})
```

4. Méthodes de test en React

Il existe 2 bibliothèques principales :

5. Tester React avec Enzyme

Enzyme permet de tester la structure du composant (HTML généré).
Il était très utilisé → aujourd'hui moins populaire mais encore enseigné.

5.1 Installation

npm install --save-dev enzyme

npm install --save-dev enzyme-adapter-react-16 --force

5.2 Exemple de composant simple

```
// App.js
import React from "react";
export default function App() {
  return (
    <div>
      <button id="ClickMe" className="click-me">Click Me</button>
    </div>
  );
```

```
}
```

5.3 Test avec Enzyme

```
// App.test.js
import React from "react";
import Enzyme, { shallow } from "enzyme";
import Adapter from "enzyme-adapter-react-16";
import App from "./App";

Enzyme.configure({ adapter: new Adapter() });

describe("Test du composant App", () => {
  it("doit afficher le bouton", () => {
    const wrapper = shallow(<App />);

    const button = wrapper.find("#ClickMe");

    expect(button).toHaveLength(1);          // Vérifie que le bouton
existe
```

```
  expect(button.text()).toEqual("Click Me"); // Vérifie le texte affiché
 });
});
```

Exécuter le test

npm test

6. Tester React avec React Testing Library (RTL) (méthode moderne)

Aujourd'hui, RTL est la bibliothèque recommandée pour tester les composants React.

Pourquoi ?

- Tests plus proches du vrai usage par un utilisateur
- Simple à apprendre
- Fonctionne sans comprendre la structure interne du composant

6.1 Installation

Avec CRA c'est déjà installé.
Sinon :

npm install @testing-library/react @testing-library/jest-dom

6.2 Reprenons le même composant

// App.js

```
import React from "react";
export default function App() {
  return (
    <div>
      <button>Click Me</button>
    </div>
  );
}
```

6.3 Test avec RTL

// App.test.js

```
import { render, screen } from "@testing-library/react";
import "@testing-library/jest-dom";
import App from "./App";

describe("Test du composant App", () => {
```

```
  it("doit afficher le bouton", () => {
    render(<App />);

    const button = screen.getByText(/click me/i);

    expect(button).toBeInTheDocument();
  });
});
```

Différences importantes (Enzyme vs RTL)

Enzyme	RTL
On teste la structure interne du composant	On teste l'usage comme un utilisateur
API complexe	API simple
Moins recommandé aujourd'hui	Recommandé par React

→ RTL est plus simple.

7. Exemple complet : Test d'interaction (RTL)

Composant :

```
import React, { useState } from "react";
```

```
export default function Counter() {
  const [count, setCount] = useState(0);

  return (
    <>
      <p>Compteur : {count}</p>
      <button onClick={() => setCount(count + 1)}>Ajouter</button>
    </>
  );
}
```

Test :

```
import { render, screen, fireEvent } from "@testing-library/react";
import Counter from "./Counter";

test("le compteur s'incrémente après le clic", () => {
  render(<Counter />);

  const button = screen.getByText(/ajouter/i);
```

```
  fireEvent.click(button);
  fireEvent.click(button);

  expect(screen.getByText("Compteur : 2")).toBeInTheDocument();
});
```

8. Résumé :

✓ On utilise Jest pour lancer les tests

✓ On teste le comportement + les informations affichées

✓ On peut tester avec :

- **Enzyme (ancien, structure interne)**
- **React Testing Library (moderne, plus simple)**

✓ Commande pour lancer les tests

npm test

OFPPT

Formateur : AARAB HAFID